

Chapitre 4 : Noe4j

Kaouther Gasmi, PhD
kaouther.gasmi@fsb.ucar.tn

Introduction

- Neo4j est un système de gestion de base de données au code source libre orienté graphes, développé en Java par la société Suédo-Américaine Neo technology
- existe depuis 2000, la version 1.0 de Neo4j est relativement récente puisque sortie en février 2010.
- **Principales caractéristiques :**
 - **Transaction** : c'est une base de données transactionnelle, respectueuse des principes ACID
 - **Haute disponibilité** : via la mise en place d'un cluster
 - **Volumétrie** : stocker et requêter des milliards de nœuds et de relations
 - **Cypher** : un langage de requête graphe déclaratif, simple et efficace
 - **Schemaless** : pas de schéma préétabli

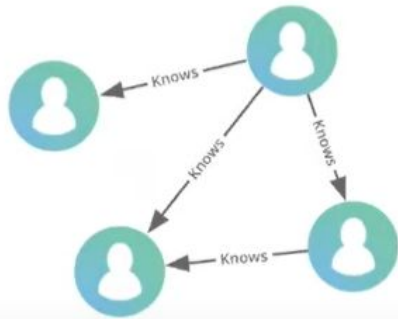
Introduction

Classement des moteurs orientés Graphes :

Rank			DBMS	Database Model	Score		
Feb 2023	Jan 2023	Feb 2022			Feb 2023	Jan 2023	Feb 2022
1.	1.	1.	Neo4j 	Graph	55.43	-0.41	-2.81
2.	2.	2.	Microsoft Azure Cosmos DB 	Multi-model 	36.51	-1.45	-3.45
3.	3.	 4.	Virtuoso 	Multi-model 	6.11	+0.23	+0.72
4.	4.	 3.	ArangoDB 	Multi-model 	5.29	+0.22	-0.11
5.	5.	5.	OrientDB	Multi-model 	4.54	+0.06	-0.49
6.	 7.	 8.	JanusGraph	Graph	2.79	+0.16	+0.44
7.	 6.	 6.	Amazon Neptune	Multi-model 	2.73	-0.08	-0.26
8.	8.	 7.	GraphDB 	Multi-model 	2.44	-0.09	-0.49
9.	9.	9.	TigerGraph	Graph	2.16	-0.04	-0.08
10.	 11.	 12.	Fauna	Multi-model 	1.89	+0.12	+0.57
11.	 10.	11.	Dgraph	Graph	1.86	+0.06	+0.14
12.	 14.	 15.	NebulaGraph 	Graph	1.79	+0.28	+0.63
13.	 17.	 20.	Memgraph 	Graph	1.68	+0.36	+1.30
14.	 12.	 10.	Stardog 	Multi-model 	1.63	+0.01	-0.35
15.	 13.	 13.	Giraph	Graph	1.59	+0.06	+0.28

Introduction

- Permet de traiter des données **massivement interconnectés**
- Ex: Domaine riche en liens entre des entités :



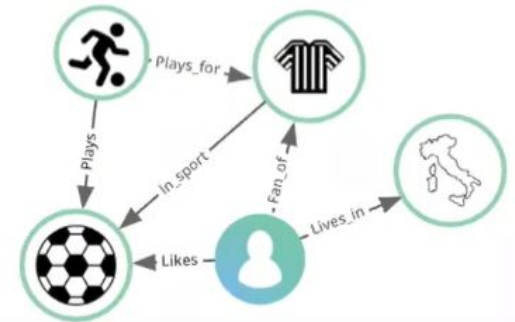
Réseau de personnes

EX: employés



Réseau de transactions

Ex : Paiement

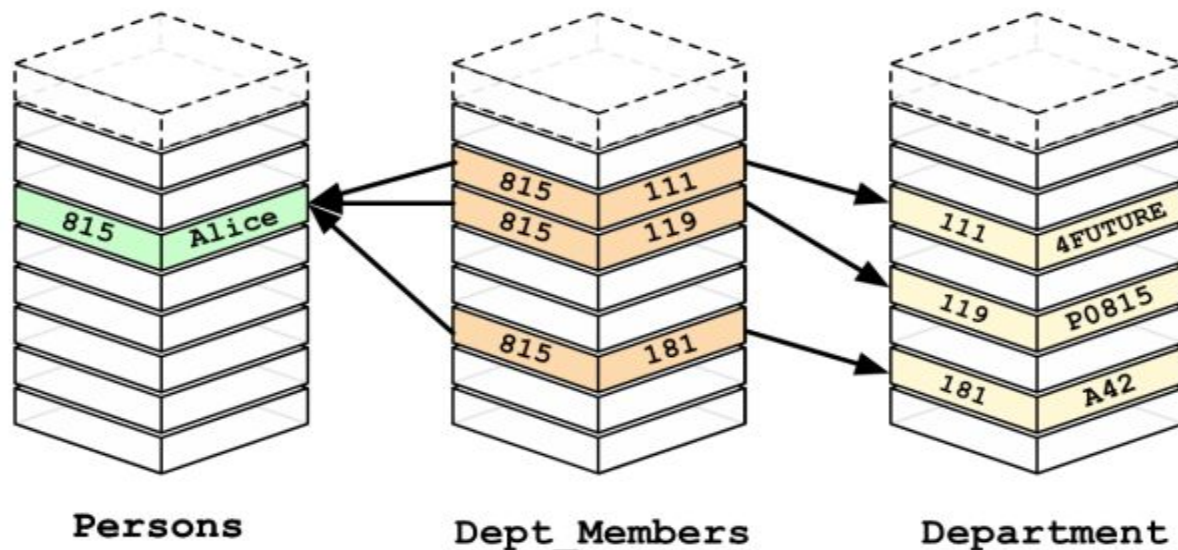


Réseau de connaissance

Ex : Sport

Relational Vs Noe4j

- Stockage et extraction de données
- Chaque nouvelle relation implique une modification du schéma de la base. Cette non-flexibilité allonge les temps de développement
- avec des données connectées, on obtient des requêtes avec beaucoup de JOIN, ce qui les rend complexes et donc difficilement maintenables
- Les bases relationnelles sont inappropriées, surtout si l'on veut faire du temps réel : schéma non flexible ; mauvaises performances ; requête complexe
- Dans un cas classique avec des relations entre deux tables :

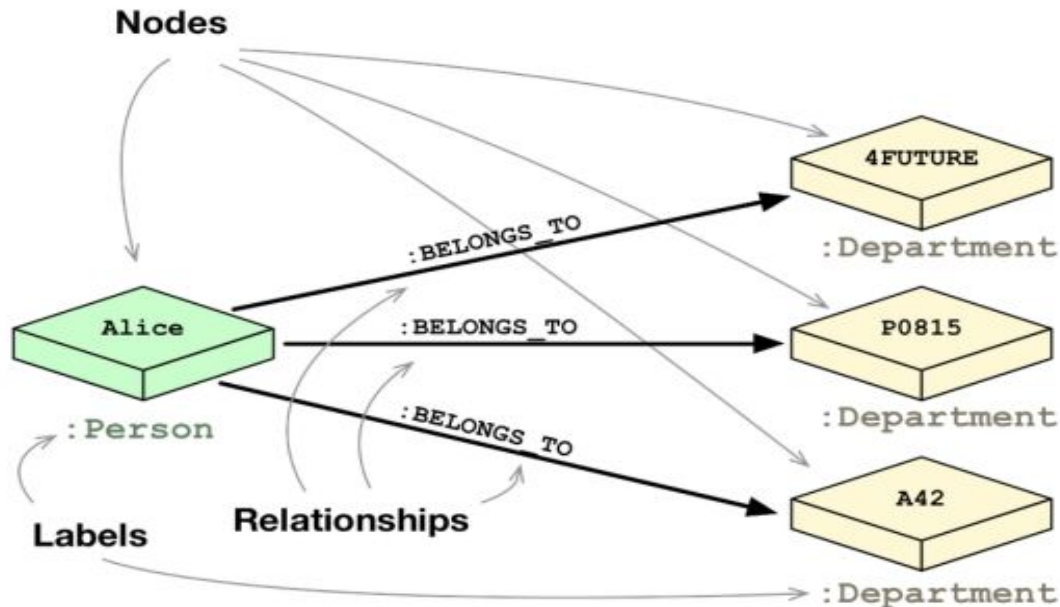


Relational Vs Noe4j

En graphe, on le modélisera comme ceci :

On retire la table qui contient les clefs étrangères.

On lie les données directement à l'aide d'une relation.

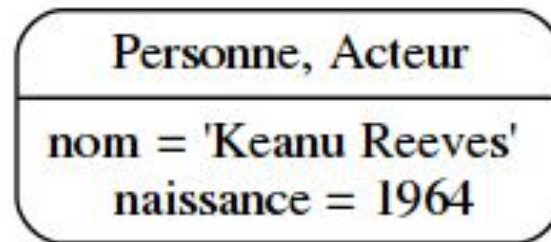


La démarche utilisée en base de données graphe permet de simplifier les modèles qui possèdent beaucoup de relations entre des données hétérogènes.

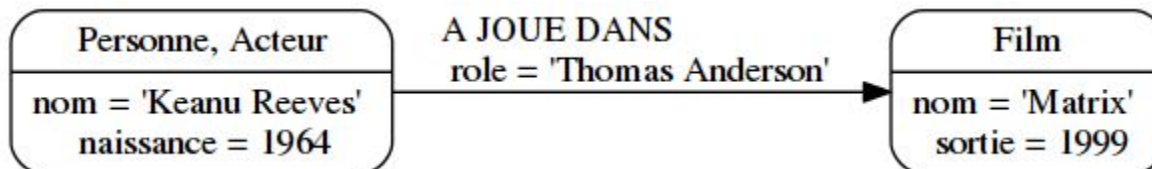
modèle de données

■ Noeuds:

- L'unité fondamentale qui forme un graphe.
- Les nœuds sont des enregistrements composés de *propriétés* de type clé/valeur, sans schéma préétabli.
- Généralement, ils représentent une entité du modèle.
- Un nœud peut avoir un *label*. Ceux-ci permettent de donner un rôle ou un type à un nœud (un nœud peut avoir plusieurs labels).

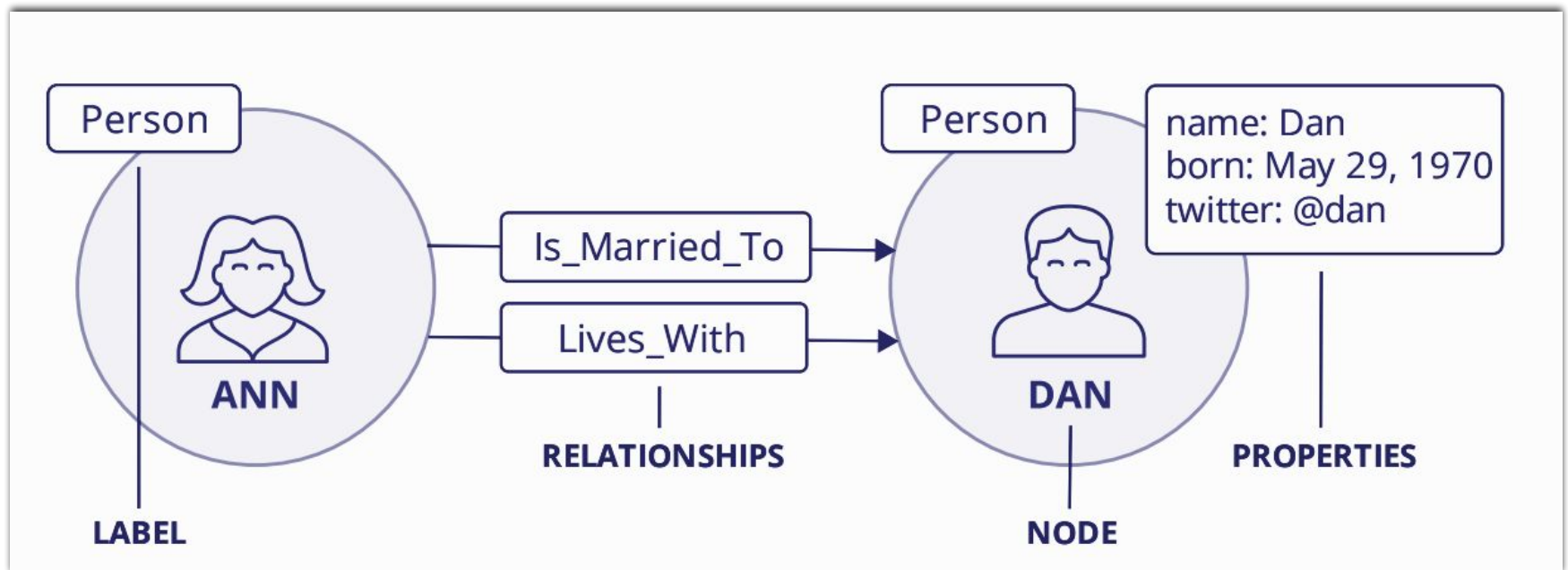


- ## ■ Relations :
- connexion entre deux nœuds avec une direction, pouvant avoir des propriétés de type clé /valeur



modèle de données

- **Propriétés: Paire cle/valeur.** Leurs types possibles correspondent aux types primitifs de Java, ou à un tableau de type primitif.
- **Exemple de graphe :**



Language Cypher

- **Cypher** : est un langage déclaratif permettant de requêter et mettre à jour le graphe. Inspiré du SQL, on y retrouve beaucoup de concepts familiers, comme les clauses WHERE, ORDER BY, SKYP, LIMIT...
- Son objectif est de permettre à l'utilisateur de définir des motifs, qui seront par la suite recherchés dans tout le graphe

- **Création d un noeud :**

```
CREATE (charlie:Person:Actor {name: 'Charlie Sheen'}),  
(oliver:Person:Director {name: 'Oliver Stone'})
```

- **Création d une relation**

Les relations sont représentées par : $\rightarrow$ on peut lui donner un nom : `-[maRelation]->`
un type : `-[maRelation:MON_TYPE]->`. Voici quelques exemples :

```
CREATE (charlie:Person:Actor {name: 'Charlie Sheen'})-[:ACTED_IN {role: 'Bud  
Fox'}]->(wallStreet:Movie {title: 'Wall  
Street'})<-[:DIRECTED]-(oliver:Person:Director {name: 'Oliver Stone'})
```

Language Cypher

■ Recherche d'un noeud :

```
MATCH (n:Person {name:'Anna'}) RETURN n.born AS  
birthYear
```

■ Clauses generales:

MATCH : recherche un élément et retourne une table ou un sous-graphe

- **DISTINCT** : élimine la redondance
- **OPTIONAL MATCH** : relation optionnelle (jointure externe en SQL)
- **WHERE** : sélection
- **RETURN** : retourne le sous-graphe ou la table

Language Cypher

■ **Clauses d'écrire:**

CREATE : crée un nœud ou une relation

- **MERGE** : crée un noeud ou une relation si elle n'existe pas déjà
- **SET** : modifie/ajoute des données/labels
- **REMOVE** : supprime des labels et des propriétés
- **DELETE** : supprime des éléments du graphe

Language Cypher

■ Recherche d'un nœud :

```
MATCH (n:Person {name:'Anna'}) RETURN n.born AS  
birthYear
```

■ Cypher - Fonctions générales utiles

id() : identifiant du nœud

labels (): le(s) label(s) du nœud

■ **type()** : le « type » de la relation (son label)

■ **length(path), relationships(path), nodes(path)**

■ **timestamp()** : une estampille temporelle

■ **upper(), lower()** : modification de la casse

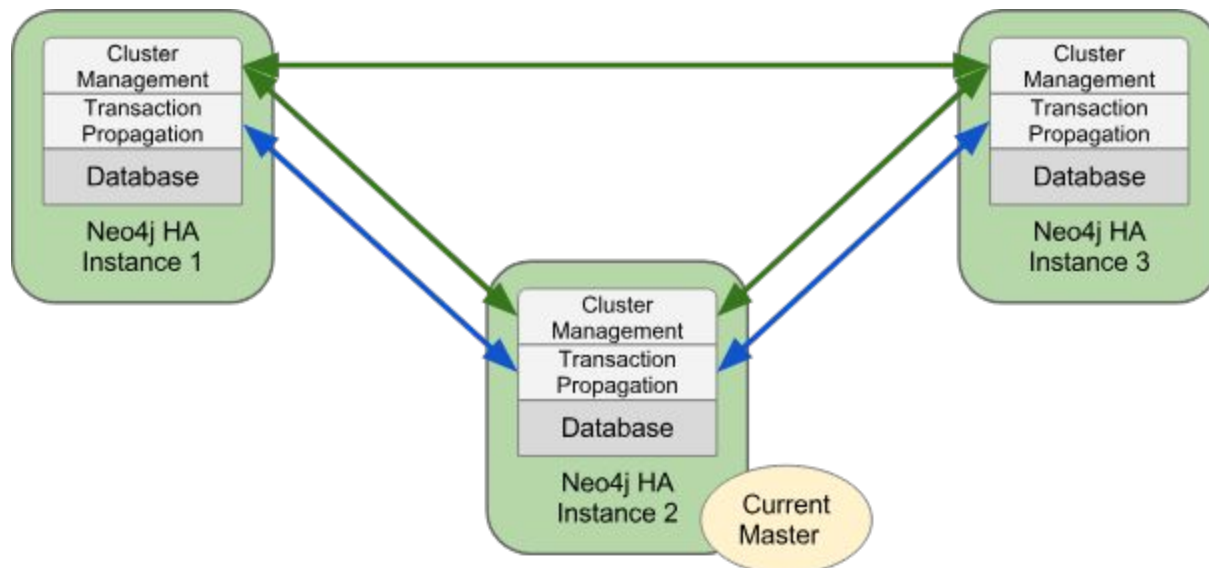
■ **range(l,u)**: retourne une liste entre l et u

Cohérence

- Isolation des opérations concurrentes jusqu'à complétion
- Tri des opérations « écriture » pour assurer un ordre de mise à jour prévisible
- Ecritures stockées dans le log de transaction de manière ordonnée
- Application des modifications dans les fichiers de données
- Changements stockés jusqu'à la fin de la transaction
- Processus de récupération : ré-application du log de transaction

Disponibilité

- Architecture maître / esclaves
- Graphe répliqué entièrement sur chaque serveur. il n'est donc pas nécessaire de rediriger les requêtes spécifiquement vers le maître
- Lecture possible sur tous les nœuds du cluster
- Taille du graphe n'impacte pas sur les performances
- Un esclave traitera les écritures en se synchronisant avec le maître pour préserver la cohérence.
- Les écritures vers le maître peuvent être poussées de manière optimiste vers 0 ou plusieurs esclaves



Forces et Faiblesses

Quand utiliser ?

- Données connectées

Quand ne pas utiliser ?

- Données peu connectées
- Mises à jour fréquentes sur une grande partie du graphe
- Analyse globale du graphe
- Quand on ne connaît pas le point d'entrée d'une requête